TD 2 : Fonctions, Allocation Dynamique et Tableaux

Exercice 1 : Arithmétique des Pointeurs

Soit :

```
int t[5] = {5, 10, 15, 20, 25};
int *p = t;
```

Donnez la valeur de `*p` et la position pointée après chaque opération (les opérations s'enchaînent) :

Opération	Initialisation (p = t)	p++	p += 2	p--	p = t + 4
Valeur de *p					
Indice pointé					

Exercice 2 : Fonctions Retournant via des Pointeurs

Écrivez une fonction `minMax(int a, int b, int c, int *min, int *max)` qui prend trois entiers et retourne via des pointeurs le minimum et le maximum des trois valeurs.

1. Écrivez le prototype et le corps de la fonction.
2. Écrivez le programme principal qui utilise cette fonction avec `a = 7`, `b = 3`, `c = 9` et affiche le min et le max.
3. Pourquoi utilise-t-on ici des pointeurs plutôt que la valeur de retour de la fonction ?

Exercice 3 : Trace d'Exécution avec Fonctions et Pointeurs

Qu'affiche le programme suivant ?

```
#include <stdio.h>
void modifier(int *x, int *y, int z) {
    *x = *x + z;
    *y = *y * z;
    z = 0;
}
int main() {
    int a = 4, b = 3, c = 2;
    modifier(&a, &b, c);    printf("a=%d  b=%d  c=%d\n", a, b, c);
    modifier(&b, &a, a);    printf("a=%d  b=%d  c=%d\n", a, b, c);
    modifier(&a, &c, c);    printf("a=%d  b=%d  c=%d\n", a, b, c);
    modifier(&b, &c, c);    printf("a=%d  b=%d  c=%d\n", a, b, c);
    return 0;
}
```

Exercice 4 : Allocation et Manipulation Dynamique

Écrivez un programme complet qui :

1. Demande à l'utilisateur un entier `n`.
2. Alloue dynamiquement un tableau de `n` entiers.

3. Remplit le tableau avec les carrés de 1 à n (i.e. `tab[i] = (i+1)*(i+1)`).
4. Affiche le tableau.
5. Libère la mémoire allouée.

Exercice 5 : Gestion dynamique complète

Considérez le programme suivant :

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n = 3;
    int *tab1 = (int *) malloc(n * sizeof(int));
    int *tab2 = (int *) calloc(n, sizeof(int));
    // Initialisation de tab1
    for (int i = 0; i < n; i++) { *(tab1+i) = (i + 1) * 10; }
    // Affichage des deux tableaux
    printf("tab1 : ");
    for (int i = 0; i < n; i++) printf("%d ", *(tab1+i));
    printf("\ntab2 : ");
    for (int i = 0; i < n; i++) printf("%d ", *(tab2+i));
    // Redimensionnement de tab1
    n = 5;
    tab1 = (int *) realloc(tab1, n * sizeof(int));
    // Ajout de nouvelles valeurs
    tab1[3] = 40;
    tab1[4] = 50;
    printf("\ntab1 apres realloc : ");
    for (int i = 0; i < n; i++) printf("%d ", *(tab1+i));
    free(tab1);
    free(tab2);
    return 0;
}
```

1. Quelles valeurs sont affichées pour `tab1` et `tab2` ? Pourquoi ?
2. Après `realloc`, les trois premières valeurs de `tab1` sont-elles conservées ? Expliquez.
3. Pourquoi est-il dangereux d'écrire directement `tab1 = realloc(tab1, ...)` sans vérification ?
4. Proposez une version sécurisée de l'appel à `realloc`.
5. Que se passe-t-il si on oublie d'appeler `free` à la fin du programme ?

Exercice 6 : Concepts Fondamentaux de l'Allocation Dynamique

Répondez aux questions suivantes :

1. Pourquoi est-il indispensable de vérifier que `malloc` ne retourne pas `NULL` ?
2. Qu'est-ce qu'une **fuite mémoire** (memory leak) ? Donnez un exemple.
3. Qu'est-ce qu'un **pointeur dangling** (dangling pointer) ? Comment l'éviter ?
4. Qu'est-ce qu'un **buffer overflow** ? Expliquez comment il peut se produire et quelles sont les conséquences possibles ?